



The Operator's Guide

Everything you need to explain, defend and run this application
— the attendance-marking layer that sits in front of
GoHighLevel.

Version 1.0 · Pre-launch

Stack Next.js 16 · TypeScript · SQLite + Drizzle ORM · Recharts

Integration GoHighLevel API v2 via Private Integration Token

Read time ~10 minutes

CONTENTS

1. What the app is for
2. The core loop
3. The three outcomes
4. What each outcome does in GHL
5. How the connection works
6. How data comes in
7. How data goes out
8. What the app infers
9. The metrics, defined
10. Safety design
11. Screen by screen
12. Design decisions
13. Limits & honest gaps
14. Path to operational
15. Cheat sheet

1 · What the app is for

GoHighLevel is a large system. Asking a salesperson to open it, find a contact, locate the right appointment, set a status, then move an opportunity to the correct pipeline stage — twelve times a day — produces two predictable failures: it takes too long, and it gets done inconsistently or not at all. When attendance data is unreliable, every number downstream is unreliable too.

FitFlow is a single screen that removes that friction. Staff see today's appointments as a list. Each one gets one click. The app then performs the full sequence of GoHighLevel updates on their behalf — correctly, in the right order, every time — and keeps a permanent record of who marked what and when.

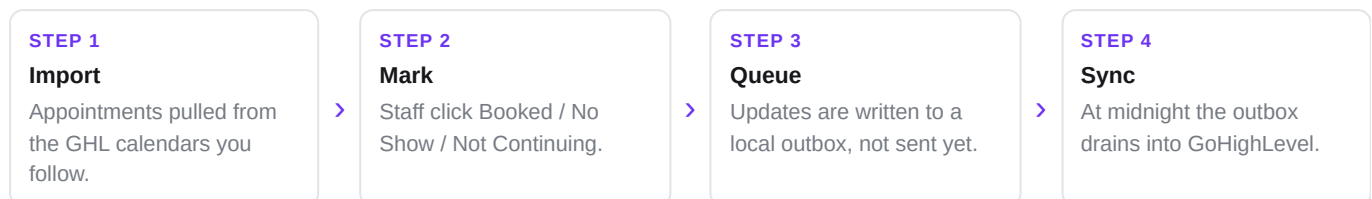
THE ONE-SENTENCE VERSION

FitFlow turns a multi-step CRM chore into a single click, and turns the resulting clicks into trustworthy pipeline data.

Why not just use GoHighLevel directly?

- **Speed.** One screen, one click, no navigation.
- **Consistency.** The stage transitions are encoded once, in software, instead of living in each rep's memory.
- **A distinction GHL doesn't have.** GoHighLevel records that someone *showed up*. It has no field for *did they commit to the next step*. That distinction is the single most valuable thing a sales team can measure, and FitFlow captures it.
- **Auditability.** Every action is written to an append-only log with an actor and timestamp.

2 · The core loop



The delay between marking and sending is deliberate, not a limitation. It buys three things:

- **Correction without consequence.** A rep who mis-clicks at 10am can fix it at 4pm. Since nothing has been sent, there is no compensating write to undo — the queued operations are simply replaced.
- **Resilience.** If GoHighLevel is down or the token has expired, staff carry on working. The queue waits.
- **Visibility.** Every send attempt is recorded with its outcome, so a failed sync is something you can see rather than something you discover weeks later.

3 · The three outcomes

The vocabulary is deliberately small. Three buttons, no free text, no ambiguity.

OUTCOME	MEANS	WHY IT'S SEPARATE
Booked	They attended <i>and</i> scheduled the next step.	The only outcome that represents forward progress. Drives the rebook rate.
No Show	They did not attend.	A recoverable state — the opportunity stays open so they can be re-engaged.
Not Continuing	They attended but are not moving forward.	Closes the opportunity as lost. Conflating this with a no-show hides <i>why</i> deals die.

THE INSIGHT THAT SHAPES THE WHOLE APP

A no-show and a not-continuing look identical in most CRMs — both are "didn't progress". But they are completely different business problems. A no-show is a **logistics** problem (reminders, timing, qualification). A not-continuing is a **sales** problem (pitch, price, fit). Separating them tells you which one you actually have.

4 · What each outcome does in GoHighLevel

One click fires an ordered sequence of API calls. The appointment status is always first and always authoritative.

STEP	BOOKED	NO SHOW	NOT CONTINUING
1. Appointment calendar event	showed	noshow	showed
2. Note audit trail	Attached to the appointment, naming the outcome, the staff member and the timestamp.		
3. Pipeline opportunity stage	Advances Consult Booked → Pre-Roadmap Booked	Moves to the matching No Show stage	Unchanged
4. Status opportunity	won if the destination is Enrolled, otherwise stays open	Stays open	lost
5. Tags contact	fitflow-attended fitflow-booked-next	fitflow-no-show	fitflow-attended fitflow-not-continuing

The tags exist so these outcomes are usable inside GoHighLevel itself — smart lists, workflow triggers, automated follow-up sequences. That is the hook for the downstream automation you'll build later.

Stage transition map

CURRENT STAGE	BOOKED →	NO SHOW →	NOT CONTINUING →
Applied	Consult Booked	—	—
Consult Booked	Pre-Roadmap Booked	Consult No Show	stays (marked lost)
Consult No Show	Pre-Roadmap Booked	Consult No Show	stays (marked lost)
Pre-Roadmap Booked	Enrolled	Roadmap No Show	Roadmap Completed: Objection
Roadmap No Show	Enrolled	Roadmap No Show	Roadmap Completed: Objection

WHY ORDER MATTERS

GoHighLevel has no transactions across endpoints. If step 3 succeeded but step 1 failed, GHL would show an opportunity that advanced for an appointment nobody attended. So step 1 is treated as authoritative: if it fails, every dependent step for that appointment is held back until the next sync run.

5 · How the connection works

FitFlow authenticates with a **Private Integration Token** — GoHighLevel's replacement for the old API keys. You create one per sub-account in *Settings* → *Private Integrations*.

Why a PIT, not OAuth

OAuth access tokens expire **daily** and require refresh-token rotation. A Private Integration token is static. For a single-location internal tool, that removes an entire category of 3am failures. OAuth only makes sense if you're distributing across sub-accounts you don't own.

Token hygiene

Shown **once** at creation. Max 5 per location. Rotate roughly every 90 days — there's a 7-day overlap window where old and new both work, so rotation is zero-downtime.

Required scopes

calendars.readonly	calendars/events.readonly
calendars/events.write	contacts.readonly
contacts.write	opportunities.readonly
opportunities.write	locations/customFields.readonly
users.readonly	

The version header trap

Every GoHighLevel request requires a `Version` header — omitting it errors. But the value is **not the same across endpoints**:

ENDPOINT FAMILY	VERSION
<code>/calendars/*</code>	<code>v3</code>
<code>/contacts/*</code> , <code>/opportunities/*</code> , <code>/users/*</code>	<code>2021-07-28</code>

FitFlow picks the header per endpoint family automatically. Sending the wrong one is the single most likely cause of a confusing 4xx error, so it is centralised in one config file rather than scattered across call sites.

Rate limits

100 requests per 10 seconds (burst) and **200,000 per day**, counted per app per location. All traffic passes through a token-bucket limiter that spaces requests and backs off automatically before the ceiling is reached.

6 · How data comes in

The import is a read-only operation. It never writes to GoHighLevel.

1. **Calendar events** — one request per followed calendar, for your chosen date range.
2. **Opportunities** — the entire pipeline is paged through in bulk (100 per page) and indexed by contact ID.
3. **Users** — one request, to translate GHL user IDs into human names.
4. **Contacts** — fetched once per unique contact, deduplicated across appointments.

A DESIGN DECISION WORTH BEING ABLE TO EXPLAIN

The obvious implementation fetches a contact *and* runs an opportunity search per appointment. For 300 appointments that's 600+ requests and an instant rate-limit failure. Fetching all opportunities in bulk and indexing them by contact turns it into roughly **3 requests plus one per unique contact**. A typical month's import lands under 60 requests.

Idempotency

Records are matched on their GoHighLevel IDs (`ghlEventId` for appointments, `ghlContactId` for leads). Re-running an import updates existing rows instead of duplicating them — so it is safe to run repeatedly, and safe to schedule.

LOCAL MARKS ALWAYS WIN

If a human has already recorded an outcome in FitFlow, a later import will **not** overwrite it. This app's vocabulary is richer than GoHighLevel's, so its own record is treated as the more authoritative one.

7 · How data goes out

Writes use the **outbox pattern**. Marking an outcome inserts rows into a local queue table; a separate process drains that queue.

QUEUE FIELD	PURPOSE
<code>operation</code>	Which of the five write types this is
<code>endpoint</code> / <code>method</code> / <code>payload</code>	The exact HTTP request, constructed at mark time
<code>sequence</code>	Ordering — 0 is the authoritative appointment write
<code>status</code> / <code>attempts</code> / <code>lastError</code>	Retry state and the reason for any failure

The midnight job is a self-rescheduling timer anchored to real local midnight in your configured timezone. It recomputes the delay after every run rather than adding a fixed 24 hours — a fixed interval drifts by an hour twice a year at daylight-saving boundaries.

8 · What the app infers

Four places where FitFlow makes an educated guess. Knowing these is what separates understanding the app from just using it.

a. Appointment type ← calendar name

GoHighLevel has **no appointment-type field**. Type is expressed purely by which calendar an appointment sits on. FitFlow reads your calendar names and maps each to one of four types (Consult, Roadmap, Follow-Up, Check-In) by keyword and name similarity. You confirm or override every mapping in Setup.

b. Pipeline stages ← name similarity

Stage IDs are account-specific UUIDs, so they cannot be hardcoded. At setup the app reads your real pipeline and scores each of its stages against FitFlow's seven canonical stages — exact match, then containment, then token overlap. Anything scoring below the confidence threshold is flagged **check** for you to confirm.

THE HIGHEST-STAKES MAPPING IN THE APP

A wrong stage mapping moves real opportunities to the wrong place in a live pipeline. This is why weak matches are surfaced rather than silently accepted, and why live mode is off by default until you've confirmed them.

c. Historical outcomes ← opportunity status

This is the most important inference to understand. GoHighLevel records `showed` — but "showed" cannot tell you whether the next step was booked. That distinction only exists in FitFlow. So when importing history:

GHL APPOINTMENT	OPPORTUNITY STATUS	IMPORTED AS
noshow	any	No Show — direct, not inferred
showed	lost / abandoned	Not Continuing — inferred
showed	won	Booked — inferred
showed	open	Left unmarked

That last row is the honest one. Guessing "Booked" would inflate your rebook rate with data GoHighLevel never actually contained. The import result tells you exactly how many outcomes were inferred.

d. Attribution — read only, never written

GoHighLevel does not expose attribution fields for writing at all. Conveniently, that means a contact update from this app **cannot** clobber first-touch attribution. Two further protections are coded in deliberately:

- Tags go through the dedicated add/remove endpoints. A `tags` array on a contact update *replaces* the entire tag set — it would silently wipe tags set by other systems.
- The `source` field is never sent. Round-tripping a fetched contact object would overwrite the original lead source.

9 · The metrics, defined

Two rates, measured against different denominators. This is the analytical heart of the app.

Show Rate

attended ÷ marked

Of everyone whose appointment was marked, how many turned up.

A logistics measure. Fix it with reminders, better qualification, shorter booking windows.

Rebook Rate

booked ÷ attended

Of everyone who turned up, how many committed to the next step.

A sales measure. Fix it with the pitch, the offer, objection handling, rep coaching.

WHY REBOOK RATE EXCLUDES NO-SHOWS

If no-shows sat in the rebook denominator, the two problems would be blended into one meaningless number. A team with great closing and terrible reminders would look identical to a team with the reverse — and the fix for each is completely different. Keeping them separate is what makes the metric actionable.

What the Metrics tab breaks down

VIEW	ANSWERS
Outcomes over time	Is volume growing, and is the mix of results shifting?
Show vs. rebook trend	Which of the two problems is moving, and in which direction?
Source effectiveness	Which lead sources produce people who actually continue — not just people who fill the calendar.
Type effectiveness	Which conversation type converts. Roadmaps should out-convert consults.
Rep performance	Rebook rate per staff member, ranked.
Pipeline distribution	Where every lead currently sits, and the value in each stage.

Sources with fewer than three marked appointments are excluded from the ranking table — a 100% rate from one appointment is noise, not a signal.

10 · Safety design

MECHANISM	WHAT IT PREVENTS
Dry run by default	Live mode must be switched on explicitly. A missing or half-finished setup can never cause an accidental write to a real account. Every operation is still built and queued — you can inspect exactly what <i>would</i> have been sent.
Provenance column	Every row is tagged <code>demo</code> , <code>ghl</code> or <code>manual</code> . A warning banner appears on every analytical page while any fabricated row exists, so sample data can never be mistaken for real performance.
Outbox + sequencing	A failed authoritative write blocks its dependent writes, so GoHighLevel is never left in a self-contradictory state.
Re-mark clears the queue	Correcting a mis-click replaces the queued operations rather than stacking a second, contradictory set on top.
Masked credentials	The API token is never returned by any endpoint — only a masked tail, so it cannot leak through a screenshot or a browser devtools panel.
Notifications off by default	<code>toNotify</code> is sent explicitly as false, so marking someone a no-show doesn't fire whatever client-facing automation is wired to that status change.

11 · Screen by screen

SCREEN	PURPOSE
Today	The reason the app exists. One day's appointments, three buttons each, a progress bar to 100%. Also holds "Sync now" and "Export Day Summary".
Dashboard	Every lead in the pipeline, filterable by stage, searchable by name, email or source.
Metrics	Trends and conversion analysis over 7/14/30/90 days.
Reports	Pipeline snapshot by stage and source, with CSV export and email delivery.
Audit	Append-only log of every mark, stage change and sync — with actor and timestamp.
Setup	Token, calendars to follow, pipeline mapping, import, and sample-data controls.
Settings	Timezone, theme, auto-sync toggle, day-summary recipient.

Export Day Summary generates the end-of-day recap and emails it via Resend. The recipient address is remembered and pre-filled next time, with the five most recent offered as suggestions. Without an API key it still generates — downloadable as CSV.

12 · Design decisions worth defending

DECISION	REASONING
Queue writes instead of sending immediately	Corrections cost nothing, outages don't block staff, and every attempt is auditable.
Three outcomes, not two	Separating "didn't show" from "showed but declined" separates a logistics problem from a sales problem.
Rebook rate measured against attendance	Mixing no-shows into the denominator produces a number that can't be acted on.
Local marks beat imported ones	FitFlow's vocabulary is richer than GHL's, so its record is the more precise one.
Timezone-aware day boundaries	GHL's API takes epoch milliseconds. Computing "today" in UTC would push evening appointments onto the wrong day's sheet.
Bulk-fetch opportunities	Turns a 600-request rate-limit failure into a ~60-request import.
Ambiguous history left unmarked	An honest gap is more valuable than a confident guess that quietly corrupts the rebook rate.
Weak stage matches flagged, not accepted	The one mistake that would damage live client data.

13 · Limits & honest gaps

LIMITATION	DETAIL
Imported history is partial	GHL can't distinguish booked from not-continuing. Rebook rate only becomes fully trustworthy after a few weeks of staff marking outcomes directly.
One-way sync	Changes made <i>inside</i> GoHighLevel don't flow back automatically. Re-run the import, or add webhooks later.
Single machine	The midnight job runs in-process. Multiple instances would each fire it. For multi-instance, move the trigger to an external cron hitting <code>POST /api/sync</code> .
Token stored in plain text	When entered through the UI it's stored unencrypted in the local database. Fine for one machine; use the environment variable for anything shared.
Untested against a live account	Written against the documented API. A few response field names are undocumented and handled defensively. Expect one or two rough edges on the first real import.
No authentication	Anyone who can reach the URL can use it. Fine on localhost; a blocker for hosting.

14 · Path to operational

Blocking — required before real use

TASK	OWNER	NOTES
Create the Private Integration Token	You	Settings → Private Integrations, with the scopes on page 3.
Run first import & verify	You	Spot-check ~10 appointments against GHL before trusting anything.
Confirm the stage mapping	You	Especially anything flagged check .
Test live mode on one appointment	Both	Turn live mode on, mark a single real appointment, confirm in GHL, then roll out.

Important — before daily team use

TASK	OWNER	NOTES
Add login	Dev	Currently no auth. Required before anyone but you uses it.
Per-user attribution	Dev	Marks currently attribute to the appointment owner, not the logged-in person.
Resend API key	You	Enables the emailed day summary. Free tier covers 3k/month.
Scheduled re-import	Dev	Newly booked appointments only appear after an import. Should run hourly.
Deploy somewhere always-on	Dev	The midnight sync only runs while the server is running.

Next phase — the downstream automation

TASK	OWNER	NOTES
Webhooks from GHL	Dev	Two-way sync so external changes flow back in.
Workflow triggers on the tags	You	The <code>fitflow-*</code> tags are the hook — build no-show recovery and objection follow-up sequences on them.
Notes / reasons on outcomes	Dev	Capture <i>why</i> someone isn't continuing. Turns the objection stage into analysable data.
Mobile layout	Dev	Marking attendance from a phone between calls.

15 · Cheat sheet

If someone asks...

"What does it actually do?"

Turns marking attendance into one click, and performs the full GoHighLevel update sequence correctly every time.

"How does it connect?"

GoHighLevel API v2 with a Private Integration Token, scoped to one sub-account.

"Why doesn't it sync instantly?"

So mis-clicks can be corrected before anything is sent, and so an outage never blocks the team.

Numbers to know

3 outcomes · **5** API calls per mark

7 pipeline stages · **4** appointment types

100 requests / 10s rate limit

~**60** requests for a typical month's import

Midnight — when the queue drains

Show rate = attended ÷ marked

Rebook rate = booked ÷ attended

Vocabulary

TERM	MEANING
PIT	Private Integration Token — GoHighLevel's replacement for API keys.
Location ID	The sub-account identifier. Every request is scoped to it.
Dry run	Operations are built and queued but never sent. The default.
Outbox	The local queue of pending GoHighLevel writes.
Provenance	Whether a row is sample data, imported, or manually entered.
Stage map	FitFlow's stage names → your account's stage UUIDs.

FitFlow — The Operator's Guide. Reflects the build as of this document's generation. The GoHighLevel integration is written against the documented v2 API and verified structurally, but has not yet been exercised against a live account — treat the first import as a supervised test.